Funkcje w PHP oraz C++ ZSE 2020

Czym jest funkcja? PHP

Funkcje w PHP są częściowo podobne do funkcji matematycznych. Mogą przyjmować argumenty oraz zwracać wartości. Nie jest to jednak wymogiem. Funkcją nazywamy napisany przez nas kod, zamknięty w nawiasy klamrowe, poprzedzony słowem kluczowym ***function*** oraz unikatową nazwą.

Funkcja bezargumentowa, nie zwracająca wartości.

Zobaczmy poniższy listing:

```
<?php
function wyswietl_powitanie() // deklaracja funkcji
{
echo "Witam serdecznie!"; // ciało funkcji, czyli
echo "Proszę się zarejestrować."; // instrukcje do wykonania
}
?>
```

Przedstawia funkcję, której zadaniem jest wyświetlenie na ekranie dwóch komunikatów. Jest to jedynie deklaracja. Żeby użyć tak napisanej funkcji należy ją wywołać. Wywołanie przedstawione jest poniżej:

```
<?php
// pozostały kod w pliku
wyswietl_powitanie();
// pozostały kod w pliku
wyswietl_powitanie();
wyswietl_powitanie();
// pozostały kod
?>
```

Jak widać, żeby użyć napisanej funkcji, wystarczy napisać jej nazwę. Jak każde polecenie w PHP, tak również wywołanie napisanej przez nas funkcji, musi być zakończone średnikiem. Co znaczą nawiasy między nazwą, a średnikiem? Jest to miejsce na podanie argumentów funkcji. Skoro nasza funkcja nie przyjmuje żadnych, są one puste. **Nie należy jednak zapominać o ich umieszczeniu, nawet gdy funkcja jest bezargumentowa!**

Funkcja w c++ // bez wartości

```
void to_jest_moja_funkcja()
{
   //INFO: tu kod Twojej funkcji
}
```

Tak wygląda najprostsza funkcja w językach C i C++. Słowo kluczowe void informuje kompilator (jak również i programistę), że funkcja nie zwraca żadnej wartości. Nazwą funkcji w tym przypadku jest to_jest_moja_funkcja. Funkcja nie przyjmuje żadnych argumentów, bowiem wewnątrz zaokrąglonych nawiasów jest pusto. Wewnątrz klamr umieszczamy kod, który ma się wykonać w chwili gdy zostanie wywołana funkcja. Jest to blok instrukcji, który zwie się ciałem funkcji.

Wywoływanie funkcji

Wywoływanie funkcji jest bardzo proste - wystarczy wpisać jej nazwę i przekazać wartości do funkcji. Ogólna postać wywołania funkcji wygląda następująco:

```
nazwa_funkcji( wartosc_argumentu_1 /*,...*/, wartosc_argumentu_n );
```

Przykład funkcji, które nie przyjmują argumentów i nie zwracają wartości

```
#include <iostream>

void moja_funkcja()
{
   std::cout << "[1] - dodawanie" << std::endl;
   std::cout << "[2] - odejmowanie" << std::endl;
   std::cout << "[0] - wyjscie z programu" << std::endl;
}

void dodawanie()
{
   std::cout << "Jeszcze nie oprogramowano" << std::endl;
}

void odejmowanie()
{
   dodawanie();
}

int main()
{
   std::cout << "W programie sa dostepne nastepujace opcje:" << std::endl;
   moja_funkcja();
   std::cout << "Zycze przyjemnego korzystania z programu" << std::endl << std::endl;
   int liczba;

do
   {
      moja_funkcja();
```

```
        std::cin >> liczba;
        switch( liczba )
        {
        case 1:
            dodawanie();
            break;
        case 2:
            odejmowanie();
            break;
            default:
            break;
        } //switch
    } while( liczba != 0 );

    return 0;
}


//KONIEC  C++
```

Funkcja zwracająca wartość

Żeby funkcja zwróciła wartość do programu głównego, musimy umieścić zwracane wyrażenie po słowie **return.**

Zobaczmy przykład:

```
<?php
function tresc_powitania() // deklaracja funkcji
{
return "Witam wszystkich!";
}
$powitanie = tresc_powitania(); //zwrot do zmiennej $powitanie
echo $powitanie;
?>
```

Przeanalizujmy inny skrypt:

```
<?php
function oblicz()
{
$zm1 = 3;
$zm1 += 5;
$zm1++;
return $zm1; //co tu zwraca funkcja
}
if (oblicz() > 5)
echo "Funkcja zwraca wartość większą od 5";
else
echo "Wartość zwracana przez funkcję jest mniejsza od 6";
?>
```

FUNKCJA W C++ np. w DEVc++

```
#include <iostream>

float dodawanie_inne()
{
   float a;
   std::cin >> a;
   float b;
   std::cin >> b;
   return a + b;
}
int main()
{
   std::cout << "Wprowadz dwie liczby: ";
   float tu_bedzie_wynik = dodawanie_inne();
   std::cout << "Wynik dodawania wynosi: " << tu_bedzie_wynik << std::endl;
   return 0;
}
```

```
Wynik:
Wprowadz dwie liczby: 4 5
Wynik dodawania wynosi: 9
--------------------------------
Process exited after 5.087 seconds with return value 0
Press any key to continue . . .
```

Funkcja przyjmująca argumenty - PHP

Co nazywamy argumentami? Wszystkie wartości przekazywane w nawiasie, zaraz po nazwie funkcji. Jest to bardzo przydatna rzecz, która w dużej mierze usprawnia programowanie. Zmienna przekazana jako argument może być używana i modyfikowana wewnątrz funkcji, bez wpływu na jej wartość w programie głównym.

```
<?php
function przywitaj($zmienna_z_imieniem)
{
echo 'Witaj '.$zmienna_z_imieniem.'!';

return $zmienna_z_imieniem;
}
$imie = "Marcin";
$wynik=przywitaj($imie);
Echo "<br>"."Wynik funkcji:  ".$wynik;
?>
```

Tworząc deklarację funkcji, podajemy jako argumenty fikcyjne nazwy zmiennych. Następnie, wywołując funkcję, podajemy istniejącą zmienną, która jest podstawiana pod zmienną fikcyjną. Podsumowując, działa to tak, że wszędzie, gdzie użyta była $zmienna_z_imieniem, użyta zostanie $imie. Funkcja wyświetli "Witaj Marcin!".

Funkcja przyjmująca argumenty - C/C++

```
int dodawanie( int a, int b )
{
   return a + b;
}


int   iWynik = dodawanie( 123, 456 );
std::cout << "Wynik dodawania wynosi: " << iWynik << std::endl;
```

Pierwszy argument funkcji (argument a) otrzyma wartość 123;
Drugi argument funkcji (argument b) otrzyma wartość 456;
Wynik funkcji dodawanie(...) zostanie zwrócony do zmiennej iWynik;
Wartość znajdująca się w zmiennej iWynik zostanie wypisana na ekran.

Sposoby przekazywania argumentów – w PHP

Argumenty funkcji mogą być przekazywane na dwa sposoby:

- Przez wartość
- Za pomocą referencji

Domyślne przekazywanie argumentów przez wartość. **Oznacza to, że w rzeczywistości funkcja otrzymuje kopię argumentów źródłowych i wszelakie operacje wykonuje na tych kopiach. Nie jest więc w stanie dokonać żadnej modyfikacji oryginału.**

Jeśli jednak chcemy mieć możliwość modyfikowania w funkcji argumentu oryginalnego, trzeba zastosować sposób drugi – **przekazywanie przez referencję**. W tym celu przed nazwą argumentu **należy umieścić znak & (ampersand).**

- za pomocą referencji (z ang. *by reference*).

Najczęściej modyfikujemy w funkcji argumentu oryginalnego. Wtedy trzeba zastosować sposób drugi — przekazywanie przez referencję. W tym celu przed nazwą argumentu należy umieścić znak & (ampersand).

```
<?php
function dodajJeden(&$liczba)  //tu jest zmiana
{
$liczba = $liczba + 1;
}
$liczba = 1;
echo("Przed wywołaniem funkcji \$liczba = $liczba \n");
dodajJeden($liczba);
echo("Po wywołaniu funkcji \$liczba = $liczba \n");
?>
```

W tym przypadku argument wejściowy od razu staje się wyjściowym.

```
<?php
function dodajJeden(&$liczba, &$liczba2)  //tu jest zmiana
{
$liczba = $liczba + 1;
$liczba2 = $liczba + 100;
}
$liczba = 1;
echo("Przed wywołaniem funkcji \$liczba = $liczba \n");
dodajJeden($liczba,$liczba2);
```

```
echo("Po wywołaniu funkcji \$liczba = $liczba \n");
echo("Po wywołaniu funkcji \$liczba2 = $liczba2 \n");
?>
```

Co pokazuje ten przykład?

Ćwiczenia praktyczne:

Zad.1

Przy pomocy przekazywania przez wartość i przekazywania przez referencję – na bazie poprzednich zadań wykonać funkcję:

- Na podstawie boku A – podanego w wywołaniu funkcji, funkcja oblicza i zwraca wyniki: - obwodu kwadratu, pola kwadratu, pola sześcianu, objętości sześcianu.

Zad2.

Przy pomocy przekazywania przez wartość i przekazywania przez referencję – na bazie poprzednich zadań wykonać funkcję:

Funkcja pobiera w wywołaniu wartości dwumianu kwadratowego – A, B, C i zwraca pierwiastki równania kwadratowego (oba) oraz zwraca wartość delty.
Gdy delta < 0 – pierwiastki zwracane równe zero.
Gdy delta = 0 – zwrot obu pierwiastków.

Domyślne argumenty funkcji

PHP pozwala na konstruowanie funkcji o domyślnych wartościach argumentów. Dzięki temu część parametrów lub nawet wszystkie będą mogły być pominięte w wywołaniu, a w ich miejsce zostaną wstawione wartości zdefiniowane podczas tworzenia funkcji

```php
<?php
function dodaj($wartosc, $ile = 1)
{
  return $wartosc + $ile;
}
$liczba1 = 10;

$liczba2 = dodaj($liczba1, 5);
echo $liczba2 , '<br />';

$liczba2 = dodaj($liczba1);
echo $liczba2 , '<br />';

$liczba2 = dodaj(5, 3);
echo $liczba2, '<br />';

$liczba2 = dodaj(5);
echo $liczba2, '<br />';

?>
```